

Flujo de trabajo del Ensamble (presentación para el equipo)

Este documento explica, con lenguaje sencillo, el flujo de trabajo que hemos seguido para construir el clasificador de basura basado en un ensamble de modelos. Está centrado en los notebooks, los scripts de `models/` y la app de Streamlit. Incluye el porqué de cada paso (split, validación, etc.), qué archivos usamos y qué artefactos genera cada fase.

1) Objetivo y contexto

- Objetivo: clasificar imágenes en 6 clases: `cardboard`, `glass`, `metal`, `paper`, `plastic`, `trash`.
- Datos: carpeta `data/raw/garbage_classification/` con una subcarpeta por clase.
- Resultado final: combinar varios modelos preentrenados (VGG16, ResNet50, MobileNetV2, EfficientNetB3) para mejorar la precisión a través de un ensamble.

2) Preparación de datos y EDA (notebooks/data-prep.ipynb)

- Exploramos el dataset: contamos imágenes por clase y visualizamos la distribución.
- Detectamos desbalance (p. ej. `trash` tiene menos ejemplos) y aplicamos data augmentation para compensar.
- Usamos `ImageDataGenerator` con rotación, desplazamientos, zoom, flips y cambios de brillo. Guardamos las imágenes aumentadas en disco y las movemos a la carpeta de su clase.

¿Por qué hacemos split (train/val/test)?

- Train: el modelo aprende a partir de estos ejemplos.
- Val (validación): nos sirve para ajustar hiperparámetros, decidir cuándo parar (early stopping), seleccionar el mejor checkpoint y comparar variantes sin "hacer trampa".
- Test: estimación final del rendimiento en datos que el modelo no ha visto durante el entrenamiento ni durante las decisiones de tuning. Es la métrica "honesta" que reportaríamos fuera del equipo.

En el notebook creamos la estructura

`data/processed/garbage_classification_split/{train,val,test}/{class}/` con proporciones 70/15/15. Por defecto usamos enlaces simbólicos (symlinks) para no duplicar imágenes; si el sistema no soporta symlinks, se puede usar `mode='copy'`.

3) Entrenamiento de modelos individuales (models/train_*.py)

- Scripts: `models/train_vgg16.py`, `models/train_resnet50.py`, `models/train_mobilenetv2.py`, `models/train_efficientnetb3.py`.
- Qué hacen:
 - Cargan el split procesado desde `data/processed/garbage_classification_split`.
 - Preprocesan las imágenes (resize y `rescale=1/255`) y aplican augmentations en `train`.
 - Construyen el backbone preentrenado (ImageNet) y añaden la cabeza de clasificación para nuestras 6 clases.

- Compilan y entrenan con callbacks: `ModelCheckpoint` (guarda el mejor .h5), `EarlyStopping` y `ReduceLROnPlateau` (si procede).
- Artefactos generados en `models/trained/`:
 - Pesos .h5 por arquitectura (p. ej. `resnet50_YYYYMMDD-HHMMSS.h5`).
 - Historial de entrenamiento .json (p. ej. `resnet50_history_YYYYMMDD-HHMMSS.json`).

Nota: Mantener consistente el tamaño de imagen y el `rescale` entre entrenamiento y evaluación/inferencia para evitar degradación de resultados.

¿Por qué usamos CNNs y no Random Forest / XGBoost / LightGBM?

- Elegimos ensamble de CNNs (VGG16, ResNet50, MobileNetV2, EfficientNetB3) porque las imágenes tienen estructura espacial: los CNN capturan patrones locales (bordes, texturas, formas) gracias al uso de convoluciones, campos receptivos y compartición de pesos. Esto es mucho más efectivo que tratar los píxeles como un vector tabular.
- Transfer learning: partimos de pesos preentrenados en ImageNet y afinamos rápido con nuestros datos. Con pocos ejemplos por clase, este “arranque” mejora la generalización.
- Modelos tabulares (Random Forest, XGBoost, LightGBM) esperan vectores fijos; si “aplanamos” la imagen, perdemos la estructura espacial y la dimensionalidad se dispara, lo que suele empeorar el rendimiento.
- Alternativa viable: CNN como extractor de características y modelo clásico encima. Es decir, usar una CNN para convertir cada imagen en un embedding (vector) y luego entrenar XGBoost/LightGBM/RandomForest sobre esos embeddings. Esto puede funcionar bien si quieres un clasificador rápido sobre buenas features o si el dataset es pequeño.
- Otra alternativa: stacking con salidas de varias CNN (probabilidades/logits) como features para un meta-modelo (por ejemplo, logística o XGBoost). Aporta mejora, pero añade complejidad y riesgo de sobreajuste si la validación no está bien separada.

4) Ensamble: qué es y cómo lo usamos (models/ensemble.py + notebooks/ensemble_analysis.ipynb)

- Un ensamble combina la salida de varios modelos para mejorar la robustez y, a menudo, la precisión. La idea es que los “errores” de un modelo se compensen con los aciertos de otros.
- Métodos que usamos:
 - Voting soft: promedio de probabilidades (recomendado). Es como preguntar a cada modelo “qué probabilidad das a cada clase” y luego hacer la media.
 - Voting hard: voto por clase más probable (menos informativo que el soft en muchos casos).
 - Bagging (promedio): similar al soft si cada modelo ya produce probabilidades.
- En `models/ensemble.py` tenemos utilidades para:
 - `build_generators(data_dir)`: construir generadores (incluye `class_indices`).
 - `load_models(model_paths)`: cargar pesos .h5 y devolver una lista (`name, model`).
 - `predict_ensemble(models, generator, method)`: combinar las predicciones según el método elegido.
- En el notebook `notebooks/ensemble_analysis.ipynb`:
 - Añadimos `../models` al `sys.path`, cargamos los modelos desde `models/trained/*.h5` y el generador de validación.
 - Ejecutamos `predict_ensemble(..., method="voting_soft")` y calculamos métricas.

¿Por qué CNNs + voting soft?

- Voting soft (promediar probabilidades) aprovecha la “confianza” de cada modelo y reduce la varianza; es más estable que el voto duro, que tira la información de probabilidad y es sensible a empates.
- Es simple de implementar y desplegar: no requiere entrenar un meta-modelo adicional y funciona bien cuando los modelos son razonablemente buenos pero tienen sesgos distintos.
- Suele mejorar macro-F1/accuracy cuando los modelos no están perfectamente calibrados pero aportan señales complementarias.
- Futuro cercano: ponderar por desempeño (weighted soft voting) o probar stacking si contamos con suficiente validación y queremos expresar décimas extra sin sobreajustar.

5) Evaluación: qué medimos y cómo interpretarlo

- Fuente de verdad: usamos el split de validación (o test si queremos la métrica final) para medir.
- Métricas principales:
 - `classification_report` (precision, recall, f1-score por clase).
 - `confusion_matrix` + `ConfusionMatrixDisplay`: muestra dónde se confunde cada clase.
- Orden de clases: obtenemos `class_names` a partir de `val_gen.class_indices` ordenado por índice. Esto es importante para alinear correctamente las probabilidades con los nombres de clase.
- ¿Cómo leer los resultados?
 - Si `f1-score` es bajo en una clase, revisar si hay pocas imágenes, si el preprocesado difiere o si esa clase es confusa (p. ej. `paper` vs `cardboard`).
 - La matriz de confusión ayuda a detectar patrones de error repetidos.

6) Inferencia y demo (app/streamlit_predict.py)

- Carga automática de modelos: la app descubre `.h5` en `models/trained/` y llama a `load_models`.
- Preprocesado en la app: `resize` y normalización `1/255` para alinear con entrenamiento.
- Mitigamos “retracing” en TensorFlow creando funciones `tf.function` por modelo.
- Obtenemos nombres de clase desde `build_generators` si existe el split procesado; si no, usamos un fallback fijo.
- Mostramos:
 - Predicción principal y tabla de probabilidades por clase.
 - Sugerencia de contenedor (mapa `BIN_MAP`) a modo orientativo.

7) Rutas y artefactos clave

- Datos crudos: `data/raw/garbage_classification/` (una carpeta por clase).
- Split procesado:
`data/processed/garbage_classification_split/{train,val,test}/{class}/`.
- Modelos entrenados: `models/trained/*.h5 + *_history_.json`.
- Código ensamble: `models/ensemble.py`.
- Notebooks:
 - `notebooks/data-prep.ipynb` (EDA, augmentación, split).
 - `notebooks/ensemble_analysis.ipynb` (carga de modelos, ensamble y métricas).
- App Streamlit: `app/streamlit_predict.py`.

8) Preguntas frecuentes (FAQs)

- ¿Qué pasa si el número de clases del modelo no coincide con las del generador?
 - La app y los notebooks hacen comprobaciones. Si difiere, mostramos aviso y, en caso necesario, mapeamos por índice. Lo ideal es regenerar el split y re-entrenar para asegurar coherencia.
- ¿Por qué la validación es separada del train?
 - Para tomar decisiones (parar, elegir hiperparámetros) sin contaminar la estimación de rendimiento. Evitamos sobreajuste al “mirar” test.
- ¿Qué método de ensamble usamos por defecto?
 - Voting soft (promedio de probabilidades), que suele ser más estable que el voto duro.
- ¿Necesito GPU para predecir?
 - No es obligatorio. Para entrenar ayuda, pero para inferencia con unos pocos modelos puede ir bien en CPU.
- ¿Puedo añadir un modelo más al ensamble?
 - Sí. Entrénalo con el mismo split y añade su `.h5` a la lista de `model_paths`.

9) Sigüientes pasos recomendados

- Métricas adicionales: curvas ROC por clase y calibración de probabilidades (p. ej. Temperature Scaling) para mejorar la interpretación.
- Ensemble ponderado: asignar más peso a los modelos con mejor validación en vez de media simple.
- Pruebas unitarias: pequeños tests para `load_models`, `predict_ensemble` y la coherencia de `class_indices`.
- Manifiesto de modelos: un `models/manifest.json` con rutas, hashes, fechas y métricas para trazar versiones.
- CI básico: lint + tests + un smoke test que cargue 1–2 imágenes y verifique que el pipeline corre.

10) Cómo reproducir (resumen práctico)

1. Preparar datos:
 - Ver `notebooks/data-prep.ipynb` para EDA, augmentación y creación del split.
2. Entrenar modelos individuales:
 - Ejecutar los scripts `models/train_*.py` (uno por arquitectura). Generan `.h5` y `*_history_.json` en `models/trained/`.
3. Evaluar el ensamble:
 - Abrir `notebooks/ensemble_analysis.ipynb`, cargar los `.h5`, construir `val_gen` y ejecutar `predict_ensemble(..., method="voting_soft")`.
4. Probar la app:
 - `streamlit run app/streamlit_predict.py` y subir una imagen.

Diagrama del pipeline (one-pager)

```

flowchart LR
    A[Datos crudos  
data/raw/garbage_classification] --> B[EDA + Augmentación  
notebooks/data-prep.ipynb]
    B --> C[Split 70/15/15  
data/processed/.../train|val|test]
    C --> D1[Entrenar VGG16  
models/train_vgg16.py]
  
```

```

    C --> D2[Entrenar ResNet50
models/train_resnet50.py]
    C --> D3[Entrenar MobileNetV2
models/train_mobilenetv2.py]
    C --> D4[Entrenar EfficientNetB3
models/train_efficientnetb3.py]
    D1 --> E[.h5 + history
models/trained/]
    D2 --> E
    D3 --> E
    D4 --> E
    E --> F[Ensamble (voting soft)
models/ensemble.py]
    F --> G[Evaluación
notebooks/ensemble_analysis.ipynb]
    E --> H[Demo Streamlit
app/streamlit_predict.py]

```

Fallback ASCII (si no se renderiza Mermaid):

```

Datos crudos -> EDA/Aug -> Split -> Entrenamiento (4 CNNs) -> .h5
                                \ \_____ /
                                Ensamble (soft)
                                Evaluación (notebook) Demo (Streamlit)

```

Slide de justificación (resumen)

- Por qué CNNs: capturan estructura espacial (bordes, texturas, formas), aprovechan transfer learning (ImageNet) y evitan tratar la imagen como vector tabular enorme sin contexto. Alternativas: usar embeddings de una CNN con XGBoost/LightGBM o hacer stacking como meta-modelo.
- Por qué voting soft: promedia probabilidades, reduce varianza y es simple de desplegar; suele mejorar frente a voto duro. Siguiendo paso natural: soft voting ponderado por desempeño o stacking con buena validación.